Vuga App: Final version of the product

Technical report
Josue Byiringiro
BSc Software Engineering (Machine Learning)
African Leadership University
Supervisor: Junior Turatsinze
2 November 2026
Repository: github.com/jbyiringiro/capstone-project
Walkthrough video: drive.google.com/file/d/11ALnAjvop92pTYnx-89AKJn1B6W2rRHp/view?usp=drive_link
Evidence: docs/testing_report.md and docs/evidence/

Vuga is an offline Kinyarwanda picture-to-speech (AAC) app for children with disabilities who cannot speak or are hard to understand. A child taps photographs to build a message, the tablet speaks it in Kinyarwanda, and an on-device language model suggests the next word. An optional FastAPI server keeps caregiver accounts (confirmed by a code texted to the caregiver's mobile number), a copy of each coded child's board settings, and the completed messages an adult chooses to sync. The evaluation is a preliminary study with adults, not children, approved by the ALU Research Ethics Committee on 2 November 2026 (SEPT26-BSE-048). This report compares the build with the proposal (revised on 23 October after supervisor feedback), then covers the implementation, code quality, algorithms and testing.

1. Scope alignment and feature validation

Planned feature (proposal)	**Intended purpose**	**Status**	**Evidence**
Implemented and tested in software			
FR4, FR6 Category board, correction, three suggestions	Compose a message by tapping, with fewer taps	**Completed**	app/src/board/; on-device KN trigram with the same top-3 as Python on 119 prefixes; browser checks 8–11
FR5 Speech in Kinyarwanda, offline	The tablet speaks for the child	**Completed**	188 clips and caregiver recordings, said as a sentence (*Ndashaka amazi*) with 1,259 generated word clips
Personal re-ranker (proposal Fig. 2)	Adapt suggestions to one child	**Completed**	The JavaScript version matches the Python model to 1e-9
FR2, FR3 Pseudonymous profiles; custom words	One board per child; words from the child's life	**Completed**	Abana and Amagambo mashya tabs; photos and voices stay on the device
NFR3 4–40 cells and 2 cm keys	Fit motor and visual abilities	**Grid configuration complete**	Physical-device and usability validation pending. Fixed pages with stable positions; keys ≥ 76 CSS px on a phone in emulation, which does not establish 2 cm targets or usability
Child mode (added)	Early communicators	**Completed**	Up to 12 pictures and a finding game
FR7, FR8 Action log, sync on request, summary	Trustworthy tap counts; data stays local	**Completed**	MessageSession, SyncService and the Incamake tab
FR1 Adult accounts with roles	Protect synchronised data	**Completed**	SMS-confirmed number; one PIN (bcrypt; locked after 5 wrong tries, counted atomically); a new PIN ends older sessions

Planned feature (proposal)	Intended purpose	Status	Evidence
NFR1 Offline; NFR4 privacy	Rural schools; children's data	**Completed**	Offline in 9/9 configurations; the API takes only codes, generated ids, word keys and typed settings, refusing names, free text and media
Model benchmark: unigram, KN, GRU, transformer	Choose the deployed predictor	**Below target**	Top-3 on split v3: 22.0, 45.8, 46.6%, under the 60% target; simulated tap saving 14.4% (target 30%). Earlier split v2 only: transformer = KN, 48.0%
NFR2 Prediction < 200 ms, audio < 300 ms	Responsive on low-end tablets	**Met in emulation**	Prediction p95 ≤ 1.9 ms; audio 92–146 ms at 4–6× CPU (Table 1)
300–500 words with 600–1,000 human recordings	Words and voices fitted to Rwanda	**Planned cut**	188 words, synthetic speech plus caregiver recordings (proposal 1.5)
FR10 Retrain and publish model versions	Improve from consented logs	**Partial**	Reproducible pipeline and ModelRegistry; no administrator interface
NFR5 English interface; FR9 talker and keyguard	Other adults; low-cost hardware	**Not built**	English dropped after design review; hardware waits for the study tablet
Pending validation with people, a physical tablet and a real phone			
Vocabulary validation (RQ1); adult study (n ≥ 20, SUS)	Suitable words; tap reduction and usability	**Approved, not run**	REC approval SEPT26-BSE-048 (valid six months); sessions 10–20 Nov; analysis scripts ready; test split sealed
Native-speaker review	Correct speech, sentences and text	**Not yet done**	188 clips, 1,259 generated words, sentence rules
Physical Android tablet; SMS to a real phone	Install, touch targets, speech start; delivery	**Not yet run**	In-app benchmark ready; the SMS client is tested against a stand-in only

Justification of changes. The final iteration closed FR3, NFR3 and the re-ranker and added child mode, requested in design review; it is a per-child option, so the study still tests the full board. The trigram is deployed: the transformer only tied it on split v2, at 338 MB, and the GRU gains 0.8 points over a trigram of about 100 KB that suggests in 0.09 ms. The study waited for ethics approval rather than being simulated with invented data, and the hardware waits for the study tablet. After review of the final build, accounts use an SMS-confirmed number and one PIN, the app's account copy was cut to coded children's words and settings, and the section tabs and suggestions are on by default (the supervisor's suggestion).

Milestone (proposal Figure 1 and objectives)	Planned	Actual
Interface, navigation, prediction, backend, database	Weeks 4–6 (9 Oct)	21 Sep, with baselines and neural models
Ethics (REC) submission and decision	Weeks 6–7 (16 Oct)	Prepared 22–23 Oct; approved 2 Nov
Testing, final iteration, model benchmark	Weeks 7–8 (23 Oct)	22 Oct; revised 23 Oct and 2 Nov
Vocabulary validation; adult study	23 Oct; 6 Nov	Planned for 10–20 Nov

Delays. The software ran ahead of the plan, but the research milestones slipped by two to three weeks: the ethics pack was prepared on 22–23 October, with the proposal revision that followed supervisor feedback, instead of in week 6. The ALU Research Ethics Committee approved it on 2 November (SEPT26-BSE-048, valid for six months), so the sessions moved to 10–20 November; until they have run, their results are pending.

2. Technical functionality and architecture

The app has four layers: the React board; plain-JavaScript domain modules (prediction, speech, symbols, the message session), tested in Node; Dexie/IndexedDB storage on the device; and an optional FastAPI service (routers, services and SQLAlchemy models on SQLite or PostgreSQL, documented in Swagger).

Front end and back end. The board needs no server: the service worker precaches the predictor, 188 photos, 188 word clips and 1,259 sentence clips, so the app works offline after the first visit. An account opens with a Rwandan mobile number and a six-digit code texted through Africa's Talking (kept as an HMAC digest, valid 10 minutes); the same PIN unlocks the device offline and signs in to the account. Each sign-in try is counted in one SQL statement before bcrypt checks the PIN, and a new PIN raises the account's token version, which ends older sessions. When an adult asks in **Ohereza**, the app uploads the completed messages of the account's own profiles, up to 1,000 per batch with every tap event; SyncService validates the whole batch before writing and treats re-sends as duplicates.

Data handling. Every task action (select, navigate, page turn, correct, speak) is logged after a draft message exists, so taps per message include Vuga. Messages with a custom word stay on the device, because its photo or label may identify a family member. The app writes children to the account copy by pseudonymous code, with their words and settings only, and the API accepts only the shapes the app writes (generated ids, word keys, typed setting values), so names, free text, photos and recordings are refused. Each save names the revision it builds on and is told the one it wrote: a device holding an older copy is refused (HTTP 409), and the caregiver sees what differs, child by child, and chooses.

3. Code quality and OOP

Principle	Where it is applied
Abstraction, polymorphism	Predictor (JavaScript) and the Python Predictor base class define scores(); KneserNeyTrigram and PersonalReranker are interchangeable; four AudioSource classes implement resolve().
Inheritance, open/closed	InstalledClipSource extends ClipSource; new model types register in ModelRegistry; ServiceError subclasses (NotFound, Conflict, StaleBackup) map to HTTP codes in one handler.
Encapsulation	MessageSession and PersonalHistory keep their state in private fields; the PIN is stored only as a PBKDF2 digest on the device and a bcrypt hash on the server.
Decorator, chain of responsibility	PersonalReranker wraps any predictor; SpeechPlanner tries each audio source in turn.
Separation of concerns, injection	Thin routers; services take an injected database session and SMS sender; 11 app feature folders with one storage module per store; tests inject fakes.

Refactoring. Dashboard.jsx (277 lines) became an 81-line shell with one file per tab, api/app/main.py (267 lines) a composition root with 7 routers and 9 services, db.js 10 storage modules, and 20 flat source files 86 files in 11 feature folders; ParentAccess.jsx became six files and the board's speech a hook; shared hooks remove repeated busy and error handling. Names are consistent, comments explain why rather than what, Ruff checks the API, and GitHub Actions run pytest, Vitest and the build on every push.

4. Algorithms and technical problem solving

- **Kneser-Ney trigram** with absolute discounting and continuation counts. The first browser port re-summed each context for every candidate word; context totals are now computed once at load, so a

suggestion is O(V): the median fell from 2.23 to 0.090 ms and p95 from 6.50 to 0.155 ms.

- **Sentence building.** Word by word, njye + gushaka + amazi is not Kinyarwanda. A rule-based realiser walks the message once, carrying the subject across clauses: subject and verb become one inflected word, a name before a request becomes a call (*Mama, ngwino hano*), and "where" agrees with the noun class (*Umusarani uri he?*). Before an object it takes the form the corpus prefers (*ndashaka amazi* 4,476 times, *nshaka* 1,505); anything uncovered falls back to word by word. All 23 study messages come out as printed on their cards.
- **Top-k and masking.** A bounded min-heap selects the top three in O(V log k), with ties broken by key so the browser and Python agree (checked against a full sort on 300 random distributions); a vocabulary mask with a floor covers enabled words the model never saw.
- **Personal re-ranker**: $\log p(s|\text{prefix}) + 0.3\cdot\log(1+\text{count}) + 0.3\cdot e^{-\Delta t/\tau} + 0.5\cdot\log(1+\text{bigram})$, then a softmax computed with log-sum-exp to avoid overflow; a history replay rebuilds the message at each selection from the event log.
- **Speech start.** At 6× CPU, profiling showed most of the delay came before playback (clip lookups, service-worker fetches, a full redraw). The board now prepares the speech while the child composes and redraws once the sound starts. The earlier 778 ms also counted 240–460 ms of the test driver's own work, which the new timing leaves out, so the two figures are not directly comparable.
- **Concurrency and sync.** PIN tries are counted by one conditional UPDATE … RETURNING (20 parallel wrong PINs: 5 checked); PIN changes and copy saves compare-and-set a version, and a sign-in re-checks it; set-based validation cut a 4,000-message sync from 12.75 s to about 5–7 s.
- **Fixed-grid paging** splits a section into pages of columns × rows and pads short pages, so keys never change size or place; rows are sized from the free screen height, so a page is always fully visible.
- **Picture-finding game**: targets are drawn by weighted sampling (more often for pictures missed before, never the same twice in a row), with an errorless-learning hint after two misses.
- **Keystroke simulator**: dynamic programming over the board's section pages, with the same totals as the study-card script.

5. Testing strategies and results

Strategy	Why and how	Result	Improvement made afterwards
Unit and reference	Pure logic decides what the child sees and hears; Vitest with Python-exported fixtures	146 passed	Showed the frozen re-ranker weights cannot surface a new custom word; documented, not tuned
Python unit and integration	Pipeline and models; the API through TestClient; hosting settings	98 passed	Each failed before its fix: a PIN reset during sign-in or a PIN change, a save told another device's revision, free text in a setting
End-to-end	Production build in headless Edge: storage, service worker, media, layout	70/70	Fixed a one-row 3×3 page and delete-like page icons; a two-device check that an older copy never replaces a newer one
Sequential batch	20 tablets × 200 messages, one request at a time	776 msg/s	Set-based validation (p95 73 ms); concurrent load is not measured
Compatibility and performance	3 engines; desktop, tablet and phone; 4× and 6× CPU with Fast 3G; offline by stopping the server	7/9 full, 2 partial	WebKit without audio reported as partial; speech prepared while composing: 146 ms at 6× CPU after optimisation and timing-method correction

Different inputs. Tested inputs include five wrong PINs on the device and for an account, twenty at once (5 checked, then locked), a session from before a PIN change, wrong or expired SMS codes, codes asked for too often, an SMS service that refuses the text, numbers that are not Rwandan, a backup save from a device with an older copy or just after another device's, a PIN reset during a sign-in or PIN change, a child's name, free text or a photo in an upload or a setting, a duplicate word, a non-image file as a photo, files over 10 MB, silent audio, a word with no audio (Erekana offered), an empty vocabulary, offline reloads, a 1,001-message backlog and invalid sync batches, rejected before any write. Every error shown is in Kinyarwanda.

Configuration	Cold (ms)	Offline	Tap p95	Speech	fps	Heap (MB)
Edge desktop	398	391	42	53	99	12.7
Edge, Galaxy Tab S4	494	329	37	53	110	12.3
Edge, Pixel 5	480	289	40	60	128	9.1
Edge, Tab S4, 4× CPU, 3G	3,129	1,179	156	92	36	7.7
Edge, Pixel 5, 6× CPU, 3G	3,479	1,797	299	146	44	19.0
Firefox desktop	743	287	28	5*	59	–
Firefox, phone size	1,070	261	30	5*	64	–
WebKit, iPad	595	890	203	–†	13‡	–
WebKit, iPhone 13	435	482	188	–†	29‡	–

Table 1. Production build on one laptop (Intel Core Ultra 7 255H), fresh profile per run, 2 November 2026; times in ms. Low-end rows use Chromium CPU throttling and Fast 3G for the cold load. Speech: from the press to the media's playing event. Prediction p95 is at most 1.9 ms in every configuration. * Firefox signals playback almost at once. † No audio output in headless WebKit on Windows; Erekana was offered. ‡ Software rendering, not representative of Safari.

Analysis. Every configuration loads, works offline, suggests words and passes the workflow; the two WebKit runs are partial. Prediction stays far inside its 200 ms budget, and speech now starts within 300 ms everywhere it was measured. On emulated low-end hardware the remaining cost is rendering (tap p95 156–299 ms, 36–44 frames a second), which points to memoised tiles and smaller photos; a first visit over Fast 3G takes 3.1–3.5 s.

6. Limits and next steps

- All measurements are headless on one laptop. A physical Android tablet (offline installation, touch targets, speech start) and a text sent to a real phone are still to be tested.
- No child or adult has used the app, and no SUS or tap-reduction result is claimed. The approved study (10–20 November) and a native speaker's review of the speech, sentences and interface text come next; the sentence rules do not yet cover object pronouns or adjective agreement.
- The two server races found in review (a PIN reset during a sign-in or PIN change, and a save told another device's revision) are fixed. Their tests force each interleaving on SQLite; PostgreSQL under concurrent load is not tested.
- Counting page turns, the study's two message sets match only on 12-key pages, so the study grid or the sets must be fixed before the sessions.
- Then: one sealed test-set evaluation, a cold-start prior so custom words can be suggested, and the HTTPS pilot in docs/deployment_plan.md.